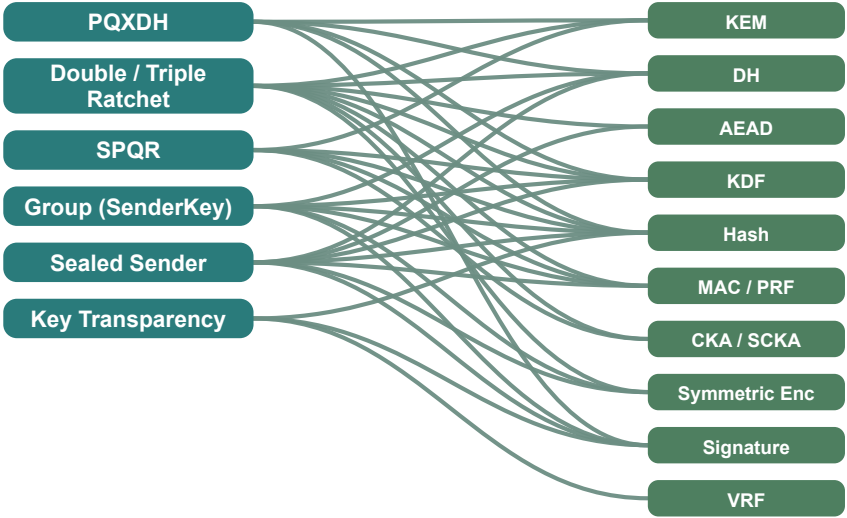
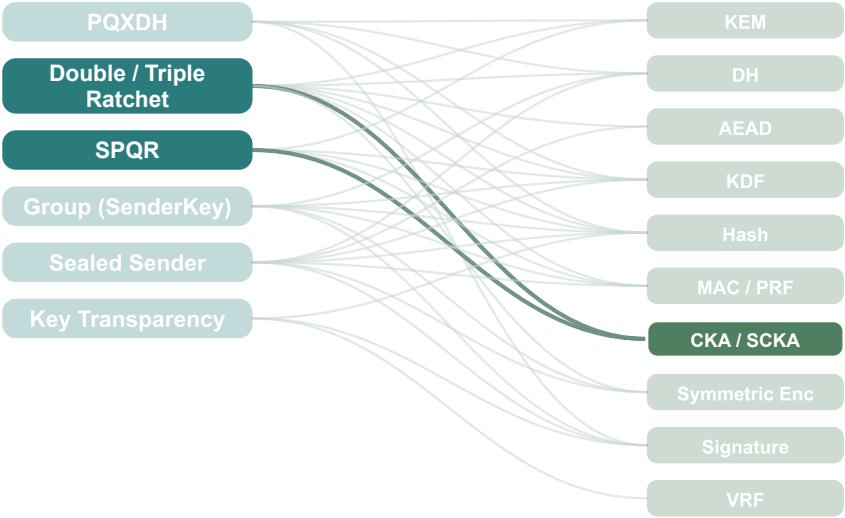
Libsignal: security protocols and crypto primitives



Libsignal: security protocols and crypto primitives



Continuous Key Agreement

A , B	participating parties
Init-KeyGen	generate shared initial key material
Init-A , Init-B	initialize local party state from shared key
Send-A , Send-B	produce a fresh key, output message, update state
Rec-A , Rec-B	receive message, derive shared key, update state

Alice($\mathsf{I}_{\mathsf{CKA}}$)		**Bob**($\mathsf{I}_{\mathsf{CKA}}$)
// generate initial state $\mathsf{st}_{\mathsf{A}} \leftarrow \mathsf{Init\text{-}A}(\mathsf{I}_{\mathsf{CKA}})$		// generate initial state $\mathsf{st}_{\mathsf{B}} \leftarrow \mathsf{Init\text{-}B}(\mathsf{I}_{\mathsf{CKA}})$
// generate new key and message $(\mathsf{K}_1, \rho_1, \mathsf{st}_{\mathsf{A}}) \leftarrow \mathsf{Send\text{-}A}(\mathsf{st}_{\mathsf{A}})$	$\xrightarrow{\rho_1}$	
		// derive shared key K_1 $(\mathsf{K}_1, \mathsf{st}_{\mathsf{B}}) \leftarrow \mathsf{Rec\text{-}B}(\mathsf{st}_{\mathsf{B}}, \rho_1)$
		// generate new key and message $(\mathsf{K}_2, \rho_2, \mathsf{st}_{\mathsf{B}}) \leftarrow \mathsf{Send\text{-}B}(\mathsf{st}_{\mathsf{B}})$
	$\xleftarrow{\rho_2}$	
// derive shared key K_2 $(\mathsf{K}_2, \mathsf{st}_{\mathsf{A}}) \leftarrow \mathsf{Rec\text{-}A}(\mathsf{st}_{\mathsf{A}}, \rho_2)$		

Alice($\mathsf{I}_{\mathsf{CKA}}$)		**Bob**($\mathsf{I}_{\mathsf{CKA}}$)
// generate initial state $\mathsf{st}_A \leftarrow$ Init-A($\mathsf{I}_{\mathsf{CKA}}$)		// generate initial state $\mathsf{st}_B \leftarrow$ Init-B($\mathsf{I}_{\mathsf{CKA}}$)
// generate new key and message (K_1, ρ_1, st_A) $\leftarrow$ Send-A(st_A)	$\xrightarrow{\rho_1}$	
		// derive shared key K_1 (K_1, st_B) $\leftarrow$ Rec-B(st_B, ρ_1)
		// generate new key and message (K_2, ρ_2, st_B) $\leftarrow$ Send-B(st_B)
	$\xleftarrow{\rho_2}$	
// derive shared key K_2 (K_2, st_A) $\leftarrow$ Rec-A(st_A, ρ_2)		
// generate new key and message (K_3, ρ_3, st_A) $\leftarrow$ Send-A(st_A)	$\xrightarrow{\rho_3}$	
		// derive shared key K_3 (K_3, st_B) $\leftarrow$ Rec-B(st_B, ρ_3)
		// generate new key and message (K_4, ρ_4, st_B) $\leftarrow$ Send-B(st_B)
	$\xleftarrow{\rho_4}$	
// derive shared key K_4 (K_4, st_A) $\leftarrow$ Rec-A(st_A, ρ_4)		

Alice($\mathsf{I}_{\mathsf{CKA}}$)		**Bob**($\mathsf{I}_{\mathsf{CKA}}$)
// generate initial state $\mathsf{st}_A \leftarrow \mathsf{Init\text{-}A}(\mathsf{I}_{\mathsf{CKA}})$		// generate initial state $\mathsf{st}_B \leftarrow \mathsf{Init\text{-}B}(\mathsf{I}_{\mathsf{CKA}})$
// generate new key and message $(K_1, \rho_1, \mathsf{st}_A) \leftarrow \mathsf{Send\text{-}A}(\mathsf{st}_A)$	$\xrightarrow{\rho_1}$	
		// derive shared key K_1 $(K_1, \mathsf{st}_B) \leftarrow \mathsf{Rec\text{-}B}(\mathsf{st}_B, \rho_1)$
		// generate new key and message $(K_2, \rho_2, \mathsf{st}_B) \leftarrow \mathsf{Send\text{-}B}(\mathsf{st}_B)$
	$\xleftarrow{\rho_2}$	
// derive shared key K_2 $(K_2, \mathsf{st}_A) \leftarrow \mathsf{Rec\text{-}A}(\mathsf{st}_A, \rho_2)$		
// generate new key and message $(K_3, \rho_3, \mathsf{st}_A) \leftarrow \mathsf{Send\text{-}A}(\mathsf{st}_A)$	$\xrightarrow{\rho_3}$	
		// derive shared key K_3 $(K_3, \mathsf{st}_B) \leftarrow \mathsf{Rec\text{-}B}(\mathsf{st}_B, \rho_3)$
		// generate new key and message $(K_4, \rho_4, \mathsf{st}_B) \leftarrow \mathsf{Send\text{-}B}(\mathsf{st}_B)$
	$\xleftarrow{\rho_4}$	
// derive shared key K_4 $(K_4, \mathsf{st}_A) \leftarrow \mathsf{Rec\text{-}A}(\mathsf{st}_A, \rho_4)$		

Security Game for CKA Def. 13 from [ACD19, *The Double Ratchet*]

O-Init(t^*)

$\mathsf{I_{CKA}} \xleftarrow{\$} \text{Init-KeyGen}(1^\lambda)$

$\mathsf{st_A} \leftarrow \text{Init-A}(\mathsf{I_{CKA}})$

$\mathsf{st_B} \leftarrow \text{Init-B}(\mathsf{I_{CKA}})$

$t_\mathsf{A}, t_\mathsf{B} \leftarrow 0$

$b \xleftarrow{\$} \{0, 1\}$

O-Send-A

t_A ++

$(\mathsf{K}_{t_\mathsf{A}}, \rho_{t_\mathsf{A}}, \mathsf{st_A}) \xleftarrow{\$} \text{Send-A}(\mathsf{st_A})$

return $(\rho_{t_\mathsf{A}}, \mathsf{K}_{t_\mathsf{A}})$

O-Send-B

t_B ++

$(\mathsf{K}_{t_\mathsf{B}}, \rho_{t_\mathsf{B}}, \mathsf{st_B}) \xleftarrow{\$} \text{Send-B}(\mathsf{st_B})$

return $(\rho_{t_\mathsf{B}}, \mathsf{K}_{t_\mathsf{B}})$

O-Rec-A

t_A ++

$(\mathsf{K}_{t_\mathsf{A}}, \mathsf{st_A}) \leftarrow \text{Rec-A}(\mathsf{st_A}, \rho_{t_\mathsf{A}})$

O-Rec-B

t_B ++

$(\mathsf{K}_{t_\mathsf{B}}, \mathsf{st_B}) \leftarrow \text{Rec-B}(\mathsf{st_B}, \rho_{t_\mathsf{B}})$

communication oracles

Security Game for CKA Def. 13 from [ACD19, *The Double Ratchet*]

O-Init(t^*)

$\mathsf{I_{CKA}} \xleftarrow{\$} \text{Init-KeyGen}(1^\lambda)$
$\mathsf{st_A} \leftarrow \text{Init-A}(\mathsf{I_{CKA}})$
$\mathsf{st_B} \leftarrow \text{Init-B}(\mathsf{I_{CKA}})$
$t_\mathsf{A},\ t_\mathsf{B} \leftarrow 0$
$b \xleftarrow{\$} \{0, 1\}$

O-Send-A

t_A ++
$(\mathsf{K}_{t_\mathsf{A}},\ \rho_{t_\mathsf{A}},\ \mathsf{st_A}) \xleftarrow{\$} \text{Send-A}(\mathsf{st_A})$
return $(\rho_{t_\mathsf{A}},\ \mathsf{K}_{t_\mathsf{A}})$

O-Send-B

t_B ++
$(\mathsf{K}_{t_\mathsf{B}},\ \rho_{t_\mathsf{B}},\ \mathsf{st_B}) \xleftarrow{\$} \text{Send-B}(\mathsf{st_B})$
return $(\rho_{t_\mathsf{B}},\ \mathsf{K}_{t_\mathsf{B}})$

O-Rec-A

t_A ++
$(\mathsf{K}_{t_\mathsf{A}},\ \mathsf{st_A}) \leftarrow \text{Rec-A}(\mathsf{st_A},\ \rho_{t_\mathsf{A}})$

O-Rec-B

t_B ++
$(\mathsf{K}_{t_\mathsf{B}},\ \mathsf{st_B}) \leftarrow \text{Rec-B}(\mathsf{st_B},\ \rho_{t_\mathsf{B}})$

O-Corr-A

req allow-corr or $\text{finished}_\mathsf{A}$
return $\mathsf{st_A}$

O-Corr-B

req allow-corr or $\text{finished}_\mathsf{B}$
return $\mathsf{st_B}$

$\text{allow-corr} \ :\Longleftrightarrow\ \max(t_\mathsf{A},\ t_\mathsf{B}) \leq t^* - 2$ $\qquad$ $\text{finished}_\mathsf{P} \ :\Longleftrightarrow\ t_\mathsf{P} \geq t^* + \Delta_\mathsf{CKA}$

communication oracles

corruption oracles

Security Game for CKA Def. 13 from [ACD19, *The Double Ratchet*]

O-Init(t^*)

$\mathsf{I_{CKA}} \xleftarrow{\$} \text{Init-KeyGen}(1^\lambda)$
$\mathsf{st_A} \leftarrow \text{Init-A}(\mathsf{I_{CKA}})$
$\mathsf{st_B} \leftarrow \text{Init-B}(\mathsf{I_{CKA}})$
$t_\mathsf{A}, t_\mathsf{B} \leftarrow 0$
$b \xleftarrow{\$} \{0,1\}$

O-Send-A

t_A ++
$(\mathsf{K}_{t_\mathsf{A}}, \rho_{t_\mathsf{A}}, \mathsf{st_A}) \xleftarrow{\$} \text{Send-A}(\mathsf{st_A})$
return $(\rho_{t_\mathsf{A}}, \mathsf{K}_{t_\mathsf{A}})$

O-Send-B

t_B ++
$(\mathsf{K}_{t_\mathsf{B}}, \rho_{t_\mathsf{B}}, \mathsf{st_B}) \xleftarrow{\$} \text{Send-B}(\mathsf{st_B})$
return $(\rho_{t_\mathsf{B}}, \mathsf{K}_{t_\mathsf{B}})$

O-Rec-A

t_A ++
$(\mathsf{K}_{t_\mathsf{A}}, \mathsf{st_A}) \leftarrow \text{Rec-A}(\mathsf{st_A}, \rho_{t_\mathsf{A}})$

O-Rec-B

t_B ++
$(\mathsf{K}_{t_\mathsf{B}}, \mathsf{st_B}) \leftarrow \text{Rec-B}(\mathsf{st_B}, \rho_{t_\mathsf{B}})$

O-Corr-A

req allow-corr or $\text{finished}_\mathsf{A}$
return $\mathsf{st_A}$

O-Corr-B

req allow-corr or $\text{finished}_\mathsf{B}$
return $\mathsf{st_B}$

O-Chall-A

t_A ++
req $t_\mathsf{A} = t^*$
$(\mathsf{K}_{t_\mathsf{A}}, \rho_{t_\mathsf{A}}, \mathsf{st_A}) \xleftarrow{\$} \text{Send-A}(\mathsf{st_A})$
if $b = 0$
  return $(\rho_{t_\mathsf{A}}, \mathsf{K}_{t_\mathsf{A}})$
else
  $\mathsf{K} \xleftarrow{\$} \mathcal{K}$
  return $(\rho_{t_\mathsf{A}}, \mathsf{K})$

O-Chall-B

t_B ++
req $t_\mathsf{B} = t^*$
$(\mathsf{K}_{t_\mathsf{B}}, \rho_{t_\mathsf{B}}, \mathsf{st_B}) \xleftarrow{\$} \text{Send-B}(\mathsf{st_B})$
if $b = 0$
  return $(\rho_{t_\mathsf{B}}, \mathsf{K}_{t_\mathsf{B}})$
else
  $\mathsf{K} \xleftarrow{\$} \mathcal{K}$
  return $(\rho_{t_\mathsf{B}}, \mathsf{K})$

$\text{allow-corr} :\Longleftrightarrow \max(t_\mathsf{A}, t_\mathsf{B}) \leq t^* - 2$ $\qquad$ $\text{finished}_\mathsf{P} :\Longleftrightarrow t_\mathsf{P} \geq t^* + \Delta_\mathsf{CKA}$

communication oracles

corruption oracles

challenge oracles

Security Game for CKA Def. 13 from [ACD19, *The Double Ratchet*]

O-Init(t^*)

$\mathsf{I_{CKA}} \xleftarrow{\$} \text{Init-KeyGen}(1^\lambda)$
$\mathsf{st_A} \leftarrow \text{Init-A}(\mathsf{I_{CKA}})$
$\mathsf{st_B} \leftarrow \text{Init-B}(\mathsf{I_{CKA}})$
$t_\mathsf{A}, t_\mathsf{B} \leftarrow 0$
$b \xleftarrow{\$} \{0,1\}$

O-Send-A

t_A ++
$(\mathsf{K}_{t_\mathsf{A}}, \rho_{t_\mathsf{A}}, \mathsf{st_A}) \xleftarrow{\$} \text{Send-A}(\mathsf{st_A})$
return $(\rho_{t_\mathsf{A}}, \mathsf{K}_{t_\mathsf{A}})$

O-Send-B

t_B ++
$(\mathsf{K}_{t_\mathsf{B}}, \rho_{t_\mathsf{B}}, \mathsf{st_B}) \xleftarrow{\$} \text{Send-B}(\mathsf{st_B})$
return $(\rho_{t_\mathsf{B}}, \mathsf{K}_{t_\mathsf{B}})$

O-Rec-A

t_A ++
$(\mathsf{K}_{t_\mathsf{A}}, \mathsf{st_A}) \leftarrow \text{Rec-A}(\mathsf{st_A}, \rho_{t_\mathsf{A}})$

O-Rec-B

t_B ++
$(\mathsf{K}_{t_\mathsf{B}}, \mathsf{st_B}) \leftarrow \text{Rec-B}(\mathsf{st_B}, \rho_{t_\mathsf{B}})$

O-Corr-A

req allow-corr or $\text{finished}_\mathsf{A}$
return $\mathsf{st_A}$

O-Corr-B

req allow-corr or $\text{finished}_\mathsf{B}$
return $\mathsf{st_B}$

O-Chall-A

t_A ++
req $t_\mathsf{A} = t^*$
$(\mathsf{K}_{t_\mathsf{A}}, \rho_{t_\mathsf{A}}, \mathsf{st_A}) \xleftarrow{\$} \text{Send-A}(\mathsf{st_A})$
if $b = 0$
  return $(\rho_{t_\mathsf{A}}, \mathsf{K}_{t_\mathsf{A}})$
else
  $\mathsf{K} \xleftarrow{\$} \mathcal{K}$
  return $(\rho_{t_\mathsf{A}}, \mathsf{K})$

O-Chall-B

t_B ++
req $t_\mathsf{B} = t^*$
$(\mathsf{K}_{t_\mathsf{B}}, \rho_{t_\mathsf{B}}, \mathsf{st_B}) \xleftarrow{\$} \text{Send-B}(\mathsf{st_B})$
if $b = 0$
  return $(\rho_{t_\mathsf{B}}, \mathsf{K}_{t_\mathsf{B}})$
else
  $\mathsf{K} \xleftarrow{\$} \mathcal{K}$
  return $(\rho_{t_\mathsf{B}}, \mathsf{K})$

$\text{Win}_\mathcal{A}$

$b' \leftarrow \mathcal{A}^\mathcal{O}$
return $(b' = b)$

$\text{allow-corr} :\iff \max(t_\mathsf{A}, t_\mathsf{B}) \leq t^* - 2$ $\qquad$ $\text{finished}_\mathsf{P} :\iff t_\mathsf{P} \geq t^* + \Delta_\mathsf{CKA}$

communication oracles

corruption oracles

challenge oracles

CKA Security against class $\mathcal{C}$:

$$\forall \mathcal{A} \in \mathcal{C}. \left|\Pr[\text{Win}_\mathcal{A}] - \frac{1}{2}\right| \leq \varepsilon$$

CKA construction

Alice(l_{CKA})		**Bob**(l_{CKA})
// generate initial state $st_A \leftarrow$ Init-A(l_{CKA})		// generate initial state $st_B \leftarrow$ Init-B(l_{CKA})
// generate new key and message (K_1, ρ_1, st_A) $\leftarrow$ Send-A(st_A)	$\xrightarrow{\rho_1}$	

		// derive shared key K_1
		(K_1, st_B) $\leftarrow$ Rec-B(st_B, ρ_1)
		// generate new key and message
		(K_2, ρ_2, st_B) $\leftarrow$ Send-B(st_B)

	$\xleftarrow{\rho_2}$	
// derive shared key K_2 (K_2, st_A) $\leftarrow$ Rec-A(st_A, ρ_2)		

Diffie-Hellman group structure and DDH assumption

Let $(G, +)$ be a cyclic group of prime order q, with generator g.

We use the **scalar multiplication** $\mathbb{Z}_q \times G \to G$:

$$x \cdot g \text{ : compute } \underbrace{g + g + \cdots + g}_{x \text{ times}}$$

Diffie-Hellman group structure and DDH assumption

Let $(G, +)$ be a cyclic group of prime order q, with generator g.

We use the **scalar multiplication** $\mathbb{Z}_q \times G \to G$:

$$x \cdot g \text{ : compute } \underbrace{g + g + \cdots + g}_{x \text{ times}}$$

Associativity $\quad x \cdot (y \cdot g) = (x \cdot y) \cdot g$

Commutativity $\quad x \cdot (y \cdot g) = y \cdot (x \cdot g)$

$\longrightarrow$ allow computing a shared key $(a \cdot b) \cdot g$

Diffie-Hellman group structure and DDH assumption

Let $(G, +)$ be a cyclic group of prime order q, with generator g.

We use the **scalar multiplication** $\mathbb{Z}_q \times G \to G$:

$$x \cdot g \text{ : compute } \underbrace{g + g + \cdots + g}_{x \text{ times}}$$

Associativity $\quad x \cdot (y \cdot g) \;=\; (x \cdot y) \cdot g$

Commutativity $\quad x \cdot (y \cdot g) \;=\; y \cdot (x \cdot g)$

$\longrightarrow$ allow computing a shared key $(a{\cdot}b) \cdot g$

DDH assumption $\quad (g,\ a{\cdot}g,\ b{\cdot}g,\ \underline{(a{\cdot}b) \cdot g}\ ) \cong (g,\ a{\cdot}g,\ b{\cdot}g,\ \underline{r \cdot g}\ )$

$\longrightarrow$ allows proving security

CKA construction from DDH

Alice($\mathsf{I}_{\mathsf{CKA}} = (h, x_0)$)	$h = x_0 \cdot g$	**Bob**($\mathsf{I}_{\mathsf{CKA}} = (h, x_0)$)
// keep group element $\mathsf{st}_A \leftarrow h$		// keep scalar $\mathsf{st}_B \leftarrow x_0$
// sample scalar, compute DH key $(K_1, \rho_1, \mathsf{st}_A) \leftarrow \begin{array}{l} x_1 \xleftarrow{\$} \mathbb{Z}_q \,; \\ (x_1 \cdot h,\ \ x_1 \cdot g,\ \ x_1) \end{array}$	$\rho_1 = x_1 \cdot g$ $\longrightarrow$	
		// compute shared DH key $(K_1, \mathsf{st}_B) \leftarrow (x_0 \cdot \rho_1,\ \ \rho_1)$
	$\mathsf{st}_A = x_1\,, \quad K_1 = (x_0 \cdot x_1) \cdot g\,, \quad \mathsf{st}_B = x_1 \cdot g$	
		// sample scalar, compute DH key $(K_2, \rho_2, \mathsf{st}_B) \leftarrow \begin{array}{l} x_2 \xleftarrow{\$} \mathbb{Z}_q \,; \\ (x_2 \cdot \mathsf{st}_B,\ \ x_2 \cdot g,\ \ x_2) \end{array}$
	$\longleftarrow$ $\rho_2 = x_2 \cdot g$	
// compute shared DH key $(K_2, \mathsf{st}_A) \leftarrow (\mathsf{st}_A \cdot \rho_2,\ \ \rho_2)$		
	$\mathsf{st}_A = x_2 \cdot g\,, \quad K_2 = (x_1 \cdot x_2) \cdot g\,, \quad \mathsf{st}_B = x_2$	

CKA construction in SPQR

CKA algo	SPQR implementation	Link
Init-KeyGen	not in SPQR *caller provides shared `auth_key`, version, direction, and chain params via `Params`*	lib.rs:54--60
Init-A	`initial_state(... direction=A2B ...)` *internal role split via `States::init_a()`*	lib.rs:210--233 states.rs:57--60
Init-B	`initial_state(... direction=B2A ...)` *internal role split via `States::init_b()`*	lib.rs:210--233 states.rs:62--64
Send-A	`send(&state, rng)` *on Alice's A2B state; internal send-side transitions in `States::send()`*	lib.rs:263--324 states.rs:115--273
Rec-B	`recv(&state, &msg)` *on Bob's B2A state; internal receive-side transitions in `States::recv()`*	lib.rs:354--453 states.rs:275--532
Send-B	`send(&state, rng)` *same public entry point, after Bob has advanced state with `Rec-B`*	lib.rs:263--324
Rec-A	`recv(&state, &msg)` *same public entry point, on Alice's evolved A2B state after `Send-B`*	lib.rs:354--453